Black Box Testing

1. EmployeeTaskController.java

This controller manages the assignment and status update of tasks related to employees. It provides functionality for retrieving tasks, marking tasks as completed, and interacting with the database for task-related data.

Functionality: Fetch Assigned Tasks

- **Test Case 1**: Fetch tasks assigned to an employee based on their ID.
    - **Input**: Employee ID = 1
    - **Expected Output**: The system should retrieve all tasks assigned to employee ID 1 and display them in the taskTable without errors.

Functionality: Mark Task as Completed

- **Test Case 2**: Mark a task as completed.
    - **Input**: Task ID = 101
    - **Expected Output**: The task with ID 101 is marked as completed in the database, and a success alert is shown to the user.

Functionality: Handle Database Error During Task Update

- **Test Case 3**: Handle a database error when attempting to complete a non-existent task.
    - **Input**: Task ID = 999 (non-existent task)
    - **Expected Output**: The system should catch a SQLException and display an error alert, indicating that the task could not be marked as completed.

---

2. LeaveRequestController.java

This controller handles the leave request process for employees, including fetching leave requests, approving, and rejecting them. It updates the leave status in the database.

Functionality: View Pending Leave Requests

- **Test Case 1**: Fetch and display all pending leave requests.
    - **Input**: None

- **Expected Output**: All pending leave requests should be fetched from the database and displayed in the leaveRequestTable.

Functionality: Approve a Leave Request

- **Test Case 2**: Approve a leave request.
  - **Input**: Leave request ID = 301
  - **Expected Output**: The leave request with ID 301 is approved in the system, and a success alert is shown.

Functionality: Reject a Leave Request

- **Test Case 3**: Reject a leave request.
  - **Input**: Leave request ID = 302
  - **Expected Output**: The leave request with ID 302 is rejected in the system, and a success alert is displayed.

Functionality: Handle No Leave Request Selected

- **Test Case 4**: Display an error when no leave request is selected during approval or rejection.
  - **Input**: No selection made by the user
  - **Expected Output**: An error alert should be displayed indicating that no leave request was selected.

---

3. MainController.java

This controller handles general employee and payroll management. It is responsible for retrieving employee data, displaying payroll details, and calculating salary with leave deductions.

Functionality: Fetch Employee Data

- **Test Case 1**: Fetch and display employee data for a specific employee.
  - **Input**: Employee ID = 1
  - **Expected Output**: Employee data for ID 1 should be retrieved from the database and displayed in the employeeTable.

Functionality: View Payroll Details

- **Test Case 2**: Fetch and display payroll details for an employee.

- **Input**: Employee ID = 1
- **Expected Output**: The payroll details for employee ID 1 should be fetched and displayed in an alert box.

Functionality: Payroll Calculation with Leave Deduction

- **Test Case 3**: Calculate payroll with deductions based on approved leaves.
  - **Input**: Employee with 10 approved leaves
  - **Expected Output**: The leave deduction is applied to the base salary, and the total pay is calculated with performance and task bonuses. The calculated payroll should be displayed in an alert box.

Functionality: Handle Database Error When Fetching Payroll

- **Test Case 4**: Handle database error when fetching payroll for a non-existent employee.
  - **Input**: Employee ID = 999 (non-existent employee)
  - **Expected Output**: The system should catch a SQLException and display an error alert indicating that payroll details could not be fetched

---

4. UpdatePerformanceFactorController.java

Functionality: Update Performance Factor

- **Test Case 1:** Update with a valid performance factor.
  - Input: performanceFactorField = "1.5"
  - Expected Output: Performance factor updated successfully.
- **Test Case 2:** Update with a non-numeric value.
  - Input: performanceFactorField = "abc"
  - Expected Output: Exception thrown or error message displayed.
- **Test Case 3:** Update with an empty field.
  - Input: performanceFactorField = ""

    - Expected Output: Exception thrown or error message displayed.
- **Test Case 4:** Update performance factor for a null employee.
    - Input: employee = null
    - Expected Output: Exception thrown or error message displayed.

---

2. UpdateEmployeeController.java

Functionality: Update Employee Details

- **Test Case 1:** Update with valid employee details.
    - Input: nameField = "John Doe", positionField = "Developer", ...
    - Expected Output: Employee details updated successfully.
- **Test Case 2:** Update with empty name field.
    - Input: nameField = "", positionField = "Developer", ...
    - Expected Output: Exception thrown or error message displayed.
- **Test Case 3:** Update with invalid salary format.
    - Input: baseSalaryField = "not_a_number"
    - Expected Output: Exception thrown or error message displayed.
- **Test Case 4:** Update for a null employee.
    - Input: employee = null
    - Expected Output: Exception thrown or error message displayed.

---

3. TaskManagementController.java

Functionality: Assign Task

- **Test Case 1:** Assign a task with valid inputs.
    - Input: taskDescriptionField = "Complete report", employeeComboBox = "1 - Alice"
    - Expected Output: Task assigned successfully.

- **Test Case 2:** Assign a task without selecting an employee.
    - Input: employeeComboBox = null
    - Expected Output: Warning message displayed: "No employee selected."
- **Test Case 3:** Assign a task with an empty description.
    - Input: taskDescriptionField = "", employeeComboBox = "1 - Alice"
    - Expected Output: Exception thrown or error message displayed.
- **Test Case 4:** Assign a task with invalid employee ID.
    - Input: employeeComboBox = "999 - Nonexistent"
    - Expected Output: Exception thrown or error message displayed.

---

4. RemoveEmployeeController.java

Functionality: Remove Employee

- **Test Case 1:** Remove an employee with a valid ID.
    - Input: idField = "1"
    - Expected Output: Employee removed successfully.
- **Test Case 2:** Remove an employee with an invalid ID.
    - Input: idField = "999"
    - Expected Output: Exception thrown or error message displayed.
- **Test Case 3:** Remove an employee with an empty ID field.
    - Input: idField = ""
    - Expected Output: Exception thrown or error message displayed.
- **Test Case 4:** Remove an employee with a non-numeric ID.
    - Input: idField = "abc"
    - Expected Output: Exception thrown or error message displayed.

---

5. PerformanceManagementController.java

Functionality: Update Performance

- **Test Case 1:** Update performance factor for a selected employee.
    - Input: performanceFactorComboBox = "Good", employeeComboBox = selected_employee
    - Expected Output: Performance factor updated successfully.
- **Test Case 2:** Update performance with no employee selected.
    - Input: employeeComboBox = null
    - Expected Output: Warning message displayed: "No employee selected."
- **Test Case 3:** Update performance with no performance factor selected.
    - Input: performanceFactorComboBox = null, employeeComboBox = selected_employee
    - Expected Output: Warning message displayed: "Please select a performance factor."
- **Test Case 4:** Update performance for an employee with a null ID.
    - Input: employeeComboBox = employee_with_null_id
    - Expected Output: Exception thrown or error message displayed.

---

6. ManagerTaskController.java

Functionality: Assign Task

- **Test Case 1:** Assign a task with valid inputs.
    - Input: taskDescriptionField = "Prepare presentation", employeeIdField = "1"
    - Expected Output: Task assigned successfully.
- **Test Case 2:** Assign a task with an empty task description.
    - Input: taskDescriptionField = "", employeeIdField = "1"
    - Expected Output: Exception thrown or error message displayed.
- **Test Case 3:** Assign a task with invalid employee ID.
    - Input: taskDescriptionField = "Prepare presentation", employeeIdField = "999"

    - Expected Output: Exception thrown or error message displayed.
- **Test Case 4:** Assign a task with a non-numeric employee ID.
    - Input: taskDescriptionField = "Prepare presentation", employeeIdField = "abc"
    - Expected Output: Exception thrown or error message displayed.

File 1: ManagerLeaveController.java

1. Handle Approve Leave Request (handleApproveLeave)

Functionality: Approves the selected leave request.

Test Cases:

- Test with a selected leave request: Ensure the leave request is approved successfully, and the status is updated.
- Test with no selected leave request: Ensure an error alert is shown when no request is selected.
- Test with a database connection error: Simulate a database failure and verify that the error alert is displayed.

2. Handle Reject Leave Request (handleRejectLeave)

Functionality: Rejects the selected leave request.

Test Cases:

- Test with a selected leave request: Ensure the leave request is rejected successfully, and the status is updated.
- Test with no selected leave request: Verify that an error alert is shown when no request is selected.
- Test with a database connection error: Simulate a database failure and verify that the error alert is displayed.

3. Update Leave Status (updateLeaveStatus)

Functionality: Updates the status of a leave request in the database.

Test Cases:

- Test with valid request ID and status: Ensure the status is correctly updated in the database.
- Test with invalid request ID: Simulate an invalid request ID and verify that the appropriate error handling occurs.

- Test with database disconnection: Simulate a database disconnection and ensure an error is logged and an alert is shown.

4. Show Alert (showAlert)

Functionality: Displays an alert to the user.

Test Cases:

- Test with valid alert inputs: Ensure the alert shows with the correct title, header, and content.
- Test with missing alert parameters: Verify behavior when some alert parameters are missing or null.
- Test alert visibility: Ensure the alert is visible on the screen and properly waits for user acknowledgment.

File 2: ManagerLeaveFunctionsController.java

1. Load Leave Requests (loadLeaveRequests)

Functionality: Loads leave requests from the database into a table view.

Test Cases:

- Test successful data retrieval: Ensure leave requests are correctly loaded into the table.
- Test with no leave requests in the database: Verify that an empty table is displayed when there are no requests.
- Test with database connection error: Simulate a database failure and ensure proper error handling occurs.

2. Handle Approve Leave Request (handleApproveLeave)

Functionality: Approves the selected leave request.

Test Cases:

- Test with a selected leave request: Ensure the leave request is approved successfully and status is updated.
- Test with no selection: Ensure an error message is shown when no request is selected.
- Test with database failure: Simulate a database failure and ensure an appropriate error message is displayed.

3. Handle Reject Leave Request (handleRejectLeave)

Functionality: Rejects the selected leave request.

Test Cases:

- Test with a selected leave request: Ensure the leave request is rejected and status is updated.
- Test with no selection: Ensure an error message is shown when no request is selected.
- Test with database failure: Simulate a database failure and ensure proper error handling**.**

4. Update Leave Status (updateLeaveStatus)

Functionality: Updates the leave request status in the database.

Test Cases:

- Test with valid request ID and status: Ensure that the status is correctly updated in the database.
- Test with an invalid request ID: Ensure appropriate error handling for invalid IDs.
- Test with database disconnection: Simulate a database failure and verify that an error message is shown

1. LeaveManagementController.java

Functionality: Load Leave Requests

- **Test Case 1:** Successfully load leave requests from the database.
    - **Input:** Valid database connection.
    - **Expected Output:** Leave requests are displayed in the table.
- **Test Case 2:** Handle database connection failure.
    - **Input:** Invalid database credentials.
    - **Expected Output:** Error alert displayed: "Database Error: Could not load leave requests."
- **Test Case 3:** No leave requests in the database.
    - **Input:** An empty leave_requests table.
    - **Expected Output:** Table remains empty.

---

2. HRManagerController.java

Functionality: Add Employee

- **Test Case 1:** Successfully add a new employee.
    - **Input:** Valid name, position, and department.
    - **Expected Output:** Employee added successfully, and the list is updated.

- **Test Case 2:** Attempt to add an employee with missing fields.
    - **Input:** Empty name field.
    - **Expected Output:** Warning alert displayed: "Missing Information. Please fill in all fields."
- **Test Case 3:** Handle database error when adding employee.
    - **Input:** Simulate a database failure.
    - **Expected Output:** Error alert displayed: "Failed to add employee."

---

3. EmployeeProfileController.java

Functionality: Set Employee Profile

- **Test Case 1:** Successfully set the employee profile.
    - **Input:** Valid Employee object.
    - **Expected Output:** All fields populated correctly.
- **Test Case 2:** Set profile with null employee.
    - **Input:** null Employee object.
    - **Expected Output:** No fields populated; no errors thrown.
- **Test Case 3:** Verify performance description based on performance factor.
    - **Input:** Employee with performanceFactor = 1.8.
    - **Expected Output:** "Very Good" displayed in the performance factor label.

---

4. EmployeeListController.java

Functionality: Load Employee Data

- **Test Case 1:** Successfully load employee data from the database.
    - **Input:** Valid database connection.
    - **Expected Output:** Employee list populated correctly.
- **Test Case 2:** Handle database connection failure.

- **Input:** Invalid database credentials.
- **Expected Output:** Error printed to console.

- **Test Case 3:** No employees in the database.
  - **Input:** An empty employees table.
  - **Expected Output:** Employee table remains empty.

---

5. EmployeeLeaveController.java

Functionality: Submit Leave Request

- **Test Case 1:** Successfully submit a leave request.
  - **Input:** Valid employee, start date, and end date.
  - **Expected Output:** Leave request submitted successfully.
- **Test Case 2:** Attempt to submit a leave request with missing fields.
  - **Input:** No employee selected.
  - **Expected Output:** Error dialog displayed: "Please fill all fields."
- **Test Case 3:** Handle database error while submitting leave request.
  - **Input:** Simulate a database failure during submission.
  - **Expected Output:** Error dialog displayed: "Failed to submit leave request."

---

6. AddEmployeeDialogController.java

Functionality: Validate Input

- **Test Case 1:** Successfully validate inputs.
  - **Input:** Valid name, position, department, and performance factor.
  - **Expected Output:** Returns true, allowing the creation of an Employee object.
- **Test Case 2:** Validate with empty fields.
  - **Input:** Empty name field.

- **Expected Output:** Error alert displayed: "No valid name!"

- **Test Case 3:** Validate with non-numeric performance factor.
    - **Input:** Performance factor set to "abc".
    - **Expected Output:** Error alert displayed: "No valid performance factor (must be a number)!"

---

7. AddEmployeeController.java

Functionality: Save New Employee

- **Test Case 1:** Successfully save a nsew employee.
    - **Input:** Valid name, position, and department.
    - **Expected Output:** Employee saved and window closed.
- **Test Case 2:** Attempt to save with empty fields.
    - **Input:** Empty position field.
    - **Expected Output:** Error alert displayed: "Please enter all fields."
- **Test Case 3:** Handle database error when saving employee.
    - **Input:** Simulate a database failure.
    - **Expected Output:** Error alert displayed: "Failed to save employee."